Búsqueda no informada

Tema 2 · Manual de estudio

Edición visual. Usa los marcadores del PDF para navegar por los conceptos. Las figuras y las secuencias de ejecución acompañan al apartado correspondiente.

1 · Del objetivo al modelo

• **Agente basado en búsqueda.** Percibe y clasifica el estado actual, fija un objetivo, anticipa los efectos de sus acciones en un modelo simbólico y busca un plan antes de ejecutarlo. El diseñador aporta el modelo y la forma de generar objetivos. Percepción y ejecución dependen del dominio (visión y robótica, o algoritmos de software).

• **Entorno de partida.** Discreto (estados y un conjunto finito de acciones y percepciones por estado), accesible (se conoce el estado relevante), estático (solo cambia cuando se actúa en este modelo) y determinista (una acción tiene un efecto definido). Por eso el caso directo del ejercicio 1.2 es **Hanoi**. Tráfico y taxi requieren afrontar cambios e incertidumbre, y cuatro en raya requiere modelar al rival.

Mundo	Modelo	Representación
Situación actual	Estado inicial s_0	Vértice de partida
Situación posible	Estado	Vértice del grafo
Acción y efecto	Operador: precondición y transición	Arco dirigido
Acciones encadenadas	Plan	Camino
Objetivo y eficiencia	meta?(s) y coste del plan	Metas y suma de pesos

Hanoi · Tres maneras de programar al agente

• **Tabla.** Guarda planes por situación. Puede aprender entradas nuevas, pero la memoria crece con los casos. Con cuatro discos, el traslado estándar tarda 15 movimientos.

• **Algoritmo específico.** Mueve n−1 discos al auxiliar, mueve el mayor al destino y pasa los n−1 del auxiliar al destino. Con n=0 termina. Parametrizar permite variar n y las agujas. Los cambios en las reglas pueden exigir otro algoritmo.

• **Búsqueda genérica.** Modela en qué aguja está cada disco, la aguja objetivo y los movimientos legales. Solo se mueve un disco libre a una aguja vacía o sobre un disco mayor. Coste 1 por movimiento. El destino es un parámetro: las diapositivas usan B en la p. 8 y C en la p. 11.

Diapositivas 4–12, 16. Solucionario, ejercicio 1. La IA del enfoque de agentes busca actuar racionalmente en su entorno (1.1b).

Hanoi · representar antes de buscar

Figuras del material docente. Lee cada panel junto a la explicación indicada.

Búsqueda como método genérico

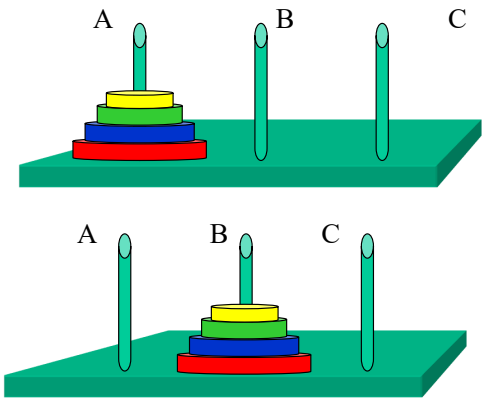
Ejemplo: Torres de Hanoi

Objetivo:

- Trasladar los discos de la aguja *A* a *B* en el mismo orden

Restricción:

- un disco mayor nunca debe reposar sobre uno de menor tamaño



¿Cómo escribir el programa de agente correspondiente?

Identifica origen, destino y orden de los discos. Un movimiento traslada únicamente el disco superior.

Solución 2: algoritmo

Algoritmos específicos del problema:

- el diseñador del agente conoce un método para resolver el problema
- codifica este método en un algoritmo *particular para el problema*
- mejorar la flexibilidad: *parametrizar* el algoritmo
- problema: el diseñador ha de *anticipar* todos los escenarios posibles
- los entornos reales suelen ser demasiado complejos como para anticipar todas las posibilidades

```
PROCEDURE MoverDiscos(n:integer;
                      origen,destino,auxiliar:char);
{ Pre: n > 0
  Post: output = [movimientos para pasar n
                  discos de la aguja origen
                  a la aguja destino] }
BEGIN
  IF n = 0 THEN   {Caso base}
    writeln
  ELSE BEGIN      {Caso recurrente}
    MoverDiscos(n-1,origen,auxiliar,destino);
    write('Pasar disco',n,'de',origen,'a',destino);
    MoverDiscos(n-1,auxiliar,destino,origen)
  END; {fin ELSE}
END; {fin MoverDiscos}
```


La recursión libera el disco mayor, lo mueve al destino y reconstruye encima la torre de n-1 discos.

Del estado a los sucesores

Modelo implícito · Lo que el algoritmo necesita

s_0 fija el punto de partida. **expandir(s)** devuelve los sucesores al aplicar los operadores permitidos. **meta?(s)** comprueba el objetivo. **c(s,s′)** da el coste del paso (positivo entero en clase). El coste del plan es la suma de los costes de sus pasos. Se pueden calcular sucesores sobre la marcha sin guardar el grafo entero.

Bloques. Solo importan las relaciones verticales y la mesa tiene espacio ilimitado. **apilar(X,Y)** mueve X sobre Y si ambos están libres y son distintos. **quitar(Y)** mueve un bloque libre a la mesa si está sobre otro bloque. En el ejemplo, C está sobre A y B está en la mesa. Para obtener A sobre B sobre C, ejecuta quitar(C), apilar(B,C) y apilar(A,B). Son tres pasos de coste 1. Primero hay que liberar A, y las otras dos acciones forman la torre.

Ejercicio 2 · Jarras de 4 y 3 litros

Estado **(x,y)** = litros en (Cuatro,Tres). Dominio $\{0,\ldots,4\} \times \{0,\ldots,3\}$, inicial (0,0), meta **x=2** con cualquier y del dominio. Cada operador cuesta 1. El dominio puede incluir estados inalcanzables. Un trasvase termina cuando se vacía el origen o se llena el destino.

Operador	Precondición	Postcondición
1 · Llenar Cuatro	$x < 4$	(4,y)
2 · Llenar Tres	$y < 3$	(x,3)
3 · Vaciar Cuatro	$x > 0$	(0,y)
4 · Vaciar Tres	$y > 0$	(x,0)
5 · Llenar Cuatro desde Tres	$x+y \geq 4$ y $x < 4$	$(4,y-(4-x))$
6 · Llenar Tres desde Cuatro	$x+y \geq 3$ y $y < 3$	$(x-(3-y),3)$
7 · Vaciar Tres en Cuatro	$x+y < 4$ y $y > 0$	(x+y,0)
8 · Vaciar Cuatro en Tres	$x+y < 3$ y $x > 0$	(0,x+y)

En 5 y 6, el origen tiene agua por las propias desigualdades. Cuando la suma iguala la capacidad del destino, se usa 5 o 6. Para implementar el trasvase en una sola regla, mueve **min(agua del origen, hueco del destino)** litros y descarta el caso de 0.

Expansiones. (0,0) genera (4,0) y (0,3). (4,0) genera (4,3), (0,0) y (1,3). (1,3) genera (4,3), (0,3), (1,0) y (4,0). Son sucesores legales antes de aplicar filtros de repetidos.

Plan de ejemplo. (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0). Operadores 2, 7, 2, 5, 3, 7. En (3,3) solo cabe 1 litro en Cuatro, así que quedan 2 en Tres. Después de vaciar Cuatro, esos 2 se trasvasan a ella.

Diapositivas 13–18. Solucionario, ejercicio 2.

Bloques · del mundo al grafo

Figuras del material docente. Lee cada panel junto a la explicación indicada.

Ejemplo: El mundo de los bloques

Situaciones:

- *n* bloques en una mesa de longitud ilimitada

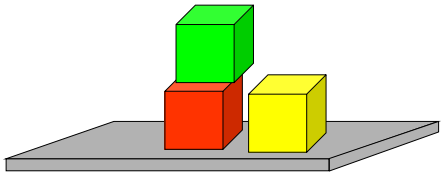
Acciones:

- *apilar*(*X*,*Y*): poner *X* encima de *Y*
  - Prec.: bloques *X* e *Y* están libres
  - Efecto: bloque *X* está encima de *Y*
- *quitar*(*Y*): poner *Y* en la mesa
  - Prec.: bloque *Y* está libre
  - Efecto: bloque *Y* está en la mesa

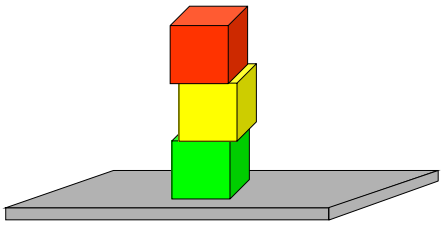
Actitud del agente:

- Objetivo: cierta configuración de bloques
- Sólo la posición *vertical* es relevante
- El coste de cada acción es uno

Situación presente



Objetivo



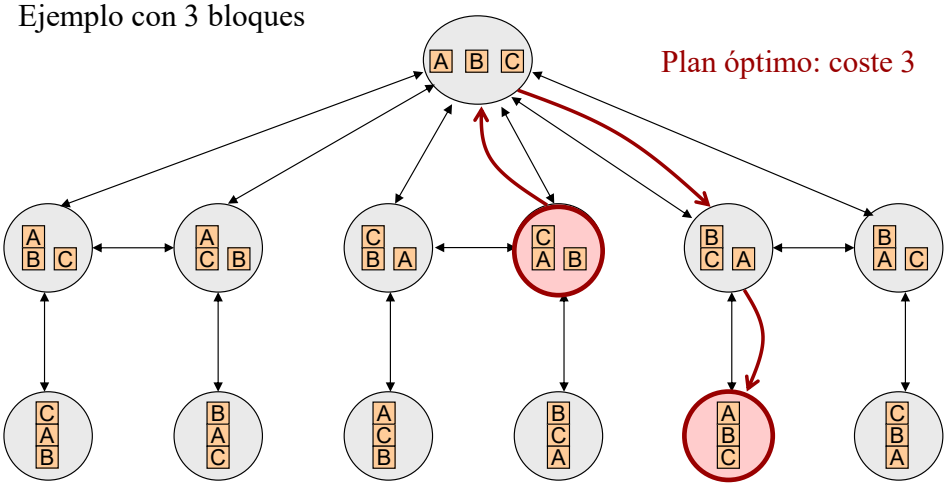
Universidad – 13 –

Solo importa qué bloque está encima de cuál. Las posiciones horizontales sobre la mesa son equivalentes.

Representación del problema de los bloques

Ejemplo con 3 bloques

Plan óptimo: coste 3



Universidad – 15 –

Sigue el camino rojo: quitar C, apilar B sobre C y apilar A sobre B. Coste total 3.

02 / ALGORITMO COMÚN

Árbol de búsqueda y abierta

Grafo de estados: describe qué situaciones y transiciones existen. Puede contener ciclos o caminos que convergen. **Árbol de búsqueda:** se construye desde s_0. Cada nodo representa un camino concreto y guarda su estado, padre, operador y, cuando interesa, profundidad y coste acumulado. Dos nodos pueden tener el mismo estado.

Ejemplo	Qué representa
Grafo: A→D→C y A→F→C	Un solo estado C alcanzable por dos rutas.
Árbol: C bajo D y C bajo F	Dos nodos, dos padres y dos planes parciales. El filtro determina si se conservan ambos.

Tus apuntes · Diapositivas 19 y 20

ABIERTA contiene las hojas pendientes. **primero** extrae la cabeza. **ordInsertar(n, abierta, orden)** fija la estrategia.

1. Introduce el nodo de s_0 en abierta.
2. Si abierta está vacía, devuelve fallo. Si tiene nodos, extrae el primero.
3. Si el nodo es meta, devuélvelo. Recupera el plan siguiendo padres y lee los operadores al revés.
4. Si no es meta, expándelo. Para cada sucesor permitido, registra padre y operador, insértalo en abierta y vuelve al paso 2.

orden	Estructura	Se extrae primero
Final	Cola FIFO · BFS	El más antiguo (menor profundidad).
Cabeza	Pila LIFO · DFS	El siguiente de la rama actual.
g ascendente	Cola de prioridad · UCS	El camino de menor coste acumulado.

Generar es crear un sucesor. **Extraer** es quitarlo de abierta. **Expandir** es calcular sus sucesores. La meta se prueba al extraer, así que el nodo meta figura en el orden de extracción y queda sin expandir.

Cómo se comparan las estrategias

Completitud: halla una solución si existe. **Optimalidad:** la solución devuelta tiene coste mínimo. **Tiempo y espacio:** cuántos nodos se procesan y cuántos deben mantenerse. Las búsquedas no informadas solo usan el modelo mínimo. La búsqueda heurística añade una estimación específica (temas 3–5). La búsqueda con estados estructurados aprovecha variables y restricciones (tema 6).

Diapositivas 19–23 y apuntes manuscritos. La comparación con A* se desarrolla en los temas siguientes.

Construcción del árbol

Figuras del material docente. Lee cada panel junto a la explicación indicada.

Metodo de busqueda

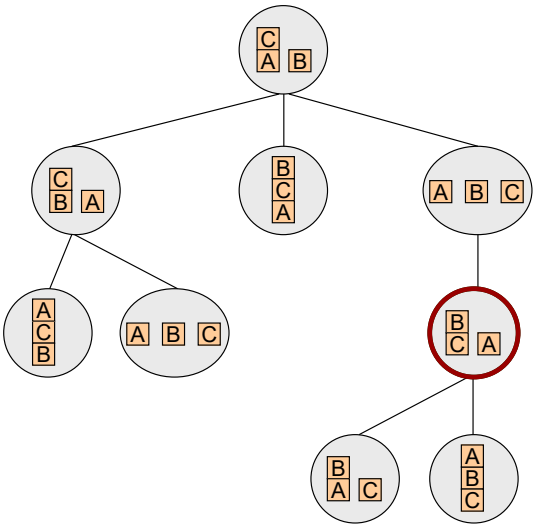
Método de búsqueda:

- *estrategia* para explorar el espacio de estados
- en cada paso se *expande* un estado
- se desarrolla sucesivamente un *árbol de búsqueda*

Método general de búsqueda:

1. seleccionar nodo hoja
2. comprobar si es nodo meta
3. *expandir* este nodo hoja

Arbol de búsqueda:



Cada expansión sustituye una hoja pendiente por sus hijos. Las apariciones repetidas de un estado representan caminos distintos.

Algoritmo de busqueda

Elementos del algoritmo

- el árbol se representa en base a un registro del tipo *nodo*
- *abierta* es una lista de nodos, con las hojas actuales del árbol
- *vacía?* determina si una lista es vacía
- *primero* quita el primer elemento de una lista
- *ordInsertar* añade un nodo a una lista, clasificado según una función de orden

```
{búsqueda general}
abierta ← s0
Repetir
    Si vacía?(abierta) entonces
        devolver(negativo)
    nodo ← primero(abierta)
    Si meta?(nodo) entonces
        devolver(nodo)
    sucesores ← expandir(nodo)
    Para cada n∈sucesores hacer
        n.padre ← nodo
        ordInsertar(n,abierta,<orden>)
Fin {repetir}
```


Seleccionar, comprobar meta y expandir. El orden de ABIERTA decide qué hoja se seleccionará después.

BFS · Primero por niveles

Extrae por la cabeza e inserta los hijos al final en el orden del enunciado. Así se agota cada nivel antes del siguiente. Con ramificación finita es **completa** si hay meta a profundidad finita. Devuelve una meta de **menor profundidad**. Con coste constante positivo por operador, también minimiza el coste. Es la observación de tus apuntes.

Ejercicio 3 · Dos caminos al mismo C

A genera D,F,G. D genera H,C^D. F genera C^f,E. H genera B y cada C genera K. G es hoja. Las metas son E y K. Conserva las dos copias de C porque el ejercicio no filtra estados repetidos.

Paso	Extrae	Añade al final	Abierta después
1	A	D F G	D F G
2	D	H C^D	F G H C^D
3	F	C^f E	G H C^D C^f E
4	G	∅	H C^D C^f E
5	H	B	C^D C^f E B
6	C^D	K^D	C^f E B K^D
7	C^f	K^f	E B K^D K^f
8	E (meta)	Se detiene	B K^D K^f

Plan: A→F→E, dos arcos. Los dos K ya se han generado al extraer E. Ambos están a profundidad 3. B, K y E no se expanden.

Coste del árbol · Ejercicio 1.3–1.5

b es el factor de ramificación y **d** la profundidad de la primera meta de BFS. En la red infinita 2D, cada nodo tiene cuatro movimientos cardinales (1.4c). Sin filtrar, el nivel k tiene $\mathbf{4^k}$ nodos (1.5b), contando la raíz como $4^0=1$. Cada nodo cuenta un camino, incluso si vuelve a (0,0).

La diapositiva 28 cuenta niveles 0…d: antes de d hay $1+b+\ldots+b^{d-1}$ nodos y, en el nivel d, se extraen desde 1 hasta b^d según dónde esté la meta. El caso medio ilustrativo usa aproximadamente la mitad del último nivel. Con $b\geq 2$ fijo, el recuento de clase es $O(b^d)$ incluso en el mejor caso. Si se contabilizan también todos los hijos de nivel d+1 generados antes de extraer la meta, el límite de tiempo y memoria en función de b y d es $O(b^{d+1})$.

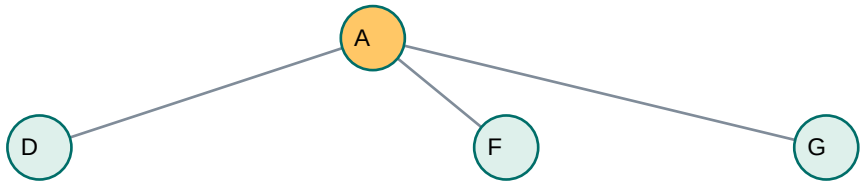
La memoria es crítica porque conserva la frontera y los datos para reconstruir caminos. Con b=10, un millón de nodos/s y 1.000 bytes por nodo, el orden de magnitud a d=10 es 10.000 millones de nodos, unas 3 horas y 10 TB.

Diapositivas 24–30. En el test 1.3 se esperan b y d según el modelo de clase. Coste constante garantiza optimalidad y la ramificación finita garantiza completitud. Medir extracciones y medir nodos generados son convenciones distintas.

BFS · ejecución visual

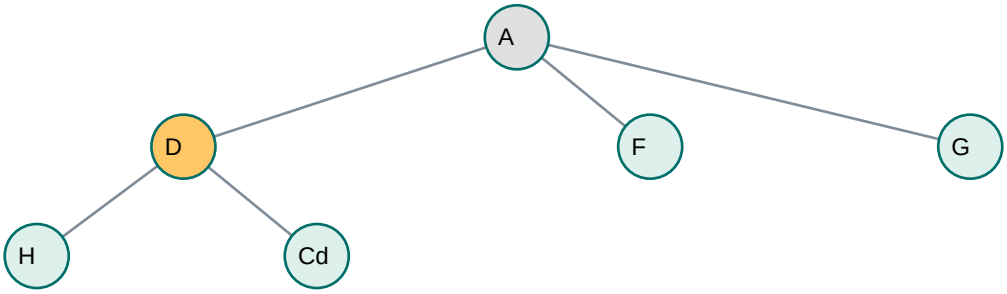
Gris: procesado. Verde claro: pendiente. Naranja: nodo del paso. Rojo: meta. C y K se duplican por sus caminos.

Paso 1 · extraer A



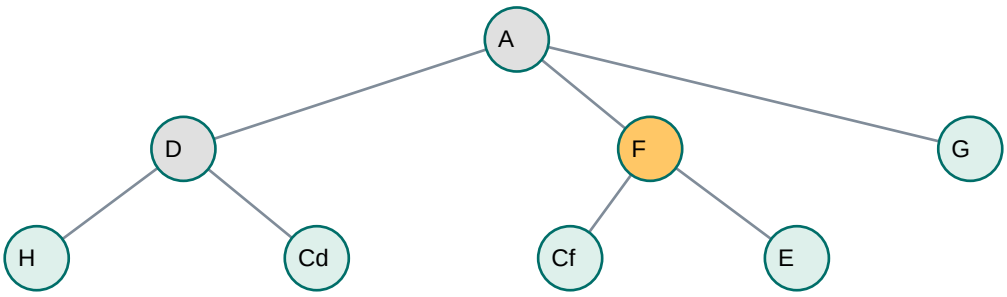
ABIERTA: D F G

Paso 2 · extraer D



ABIERTA: F G H Cd

Paso 3 · extraer F

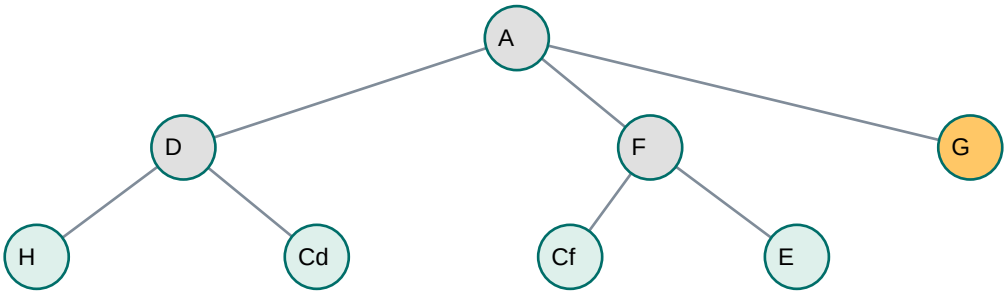


ABIERTA: G H Cd Cf E

BFS · ejecución visual

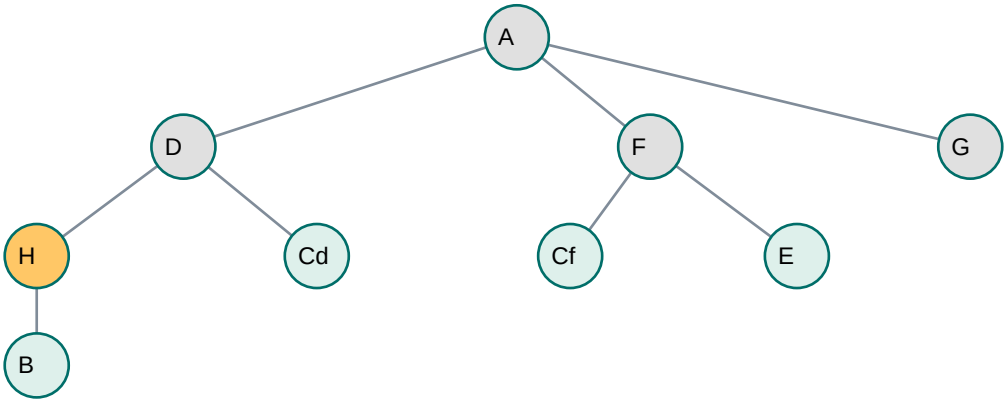
Gris: procesado. Verde claro: pendiente. Naranja: nodo del paso. Rojo: meta. C y K se duplican por sus caminos.

Paso 4 · extraer G



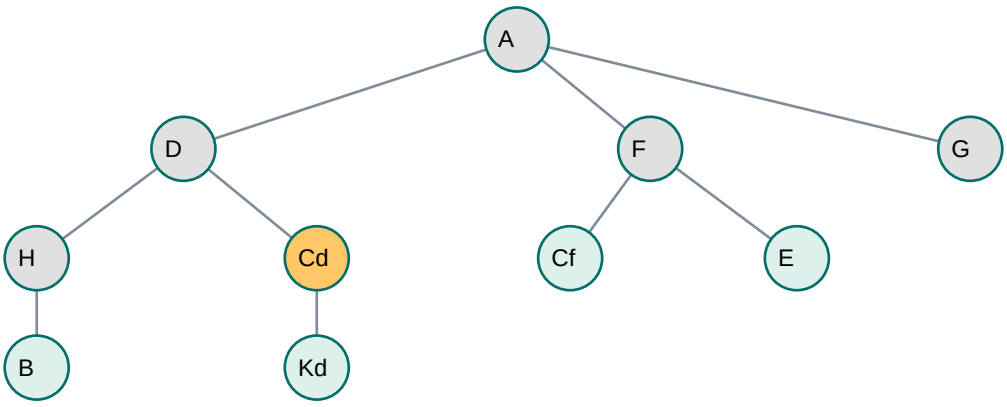
ABIERTA: H Cd Cf E

Paso 5 · extraer H



ABIERTA: Cd Cf E B

Paso 6 · extraer Cd

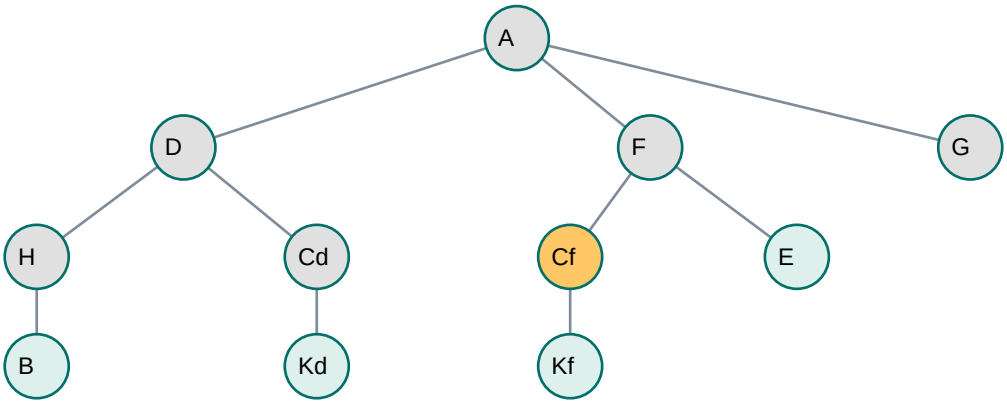


ABIERTA: Cf E B Kd

BFS · ejecución visual

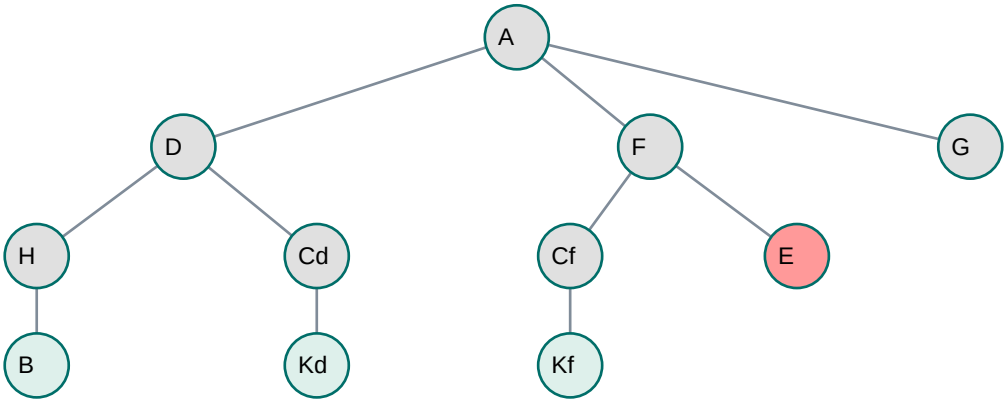
Gris: procesado. Verde claro: pendiente. Naranja: nodo del paso. Rojo: meta. C y K se duplican por sus caminos.

Paso 7 · extraer Cf



ABIERTA: E B Kd Kf

Paso 8 · extraer E

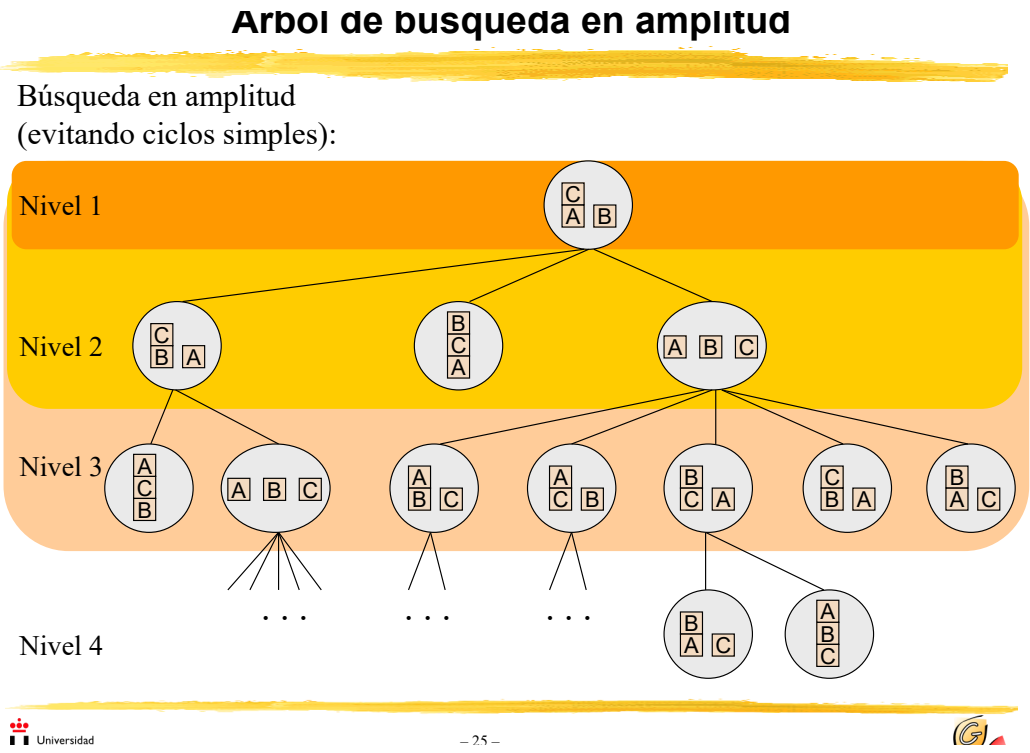


ABIERTA: B Kd Kf

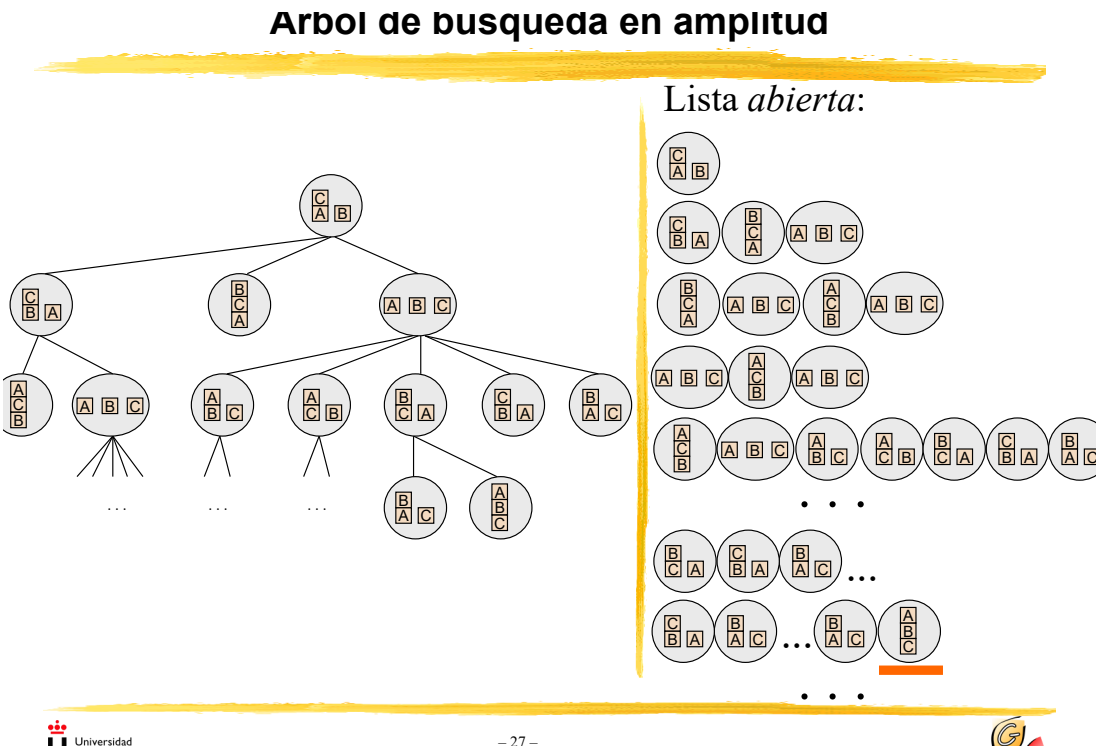
Meta aceptada. Se reconstruye el camino mediante los padres.

Amplitud · niveles y frontera

Figuras del material docente. Lee cada panel junto a la explicación indicada.



La búsqueda avanza por niveles. Evitar el retorno al padre conserva otros caminos repetidos.



Compara las hojas pendientes del árbol con la lista ABIERTA. Los hijos se añaden detrás de los nodos que ya esperaban.

DFS · Primero por ramas

Extrae de cabeza e inserta también por cabeza. Si el orden de visita debe ser H antes de C, antepón el bloque **[H,C]** o apila primero C y luego H. Insertar H y después C individualmente por cabeza invierte el orden.

Ejercicio 4 · Mismo grafo, distinta solución

Paso	Extrae	Abierta después
1	A	D F G
2	D	H C^D F G
3	H	B C^D F G
4	B	C^D F G
5	C^D	K^D F G
6	K^D (meta)	F G

B agota la primera rama. La siguiente hoja pendiente es C^D y su hijo K se extrae antes de volver a F. DFS devuelve **A→D→C→K** de profundidad 3 aunque existe E a profundidad 2. Con costes unitarios, esta traza demuestra que DFS puede devolver un plan más caro.

Completitud, tiempo y memoria

• **Completitud.** Sin filtro, puede quedarse en una rama infinita aunque haya una meta en otra. Con ramificación finita y límite d* recorre un árbol finito y encuentra las metas alcanzables dentro del límite. Si el espacio alcanzable es finito y se filtran todos los repetidos, también termina y encuentra una meta alcanzable.

• **Tiempo.** Con límite d*, en el peor caso examina $1+b+\ldots+b^{d*}$ nodos, $O(b^{d*})$ para $b\geq 2$. En el mejor caso, llega a una meta de profundidad d siguiendo su rama, O(d) expansiones u O(bd) nodos generados si se crean todos los hermanos.

• **Espacio.** Si se retiene todo el árbol, el peor caso puede ser exponencial, como en la implementación que comenta el solucionario. Si se liberan ramas agotadas y se guardan el camino actual y sus hermanos pendientes, basta O(bm), con m la profundidad máxima recorrida ($\leq$d* si hay límite). Un registro global de visitados aumenta la memoria.

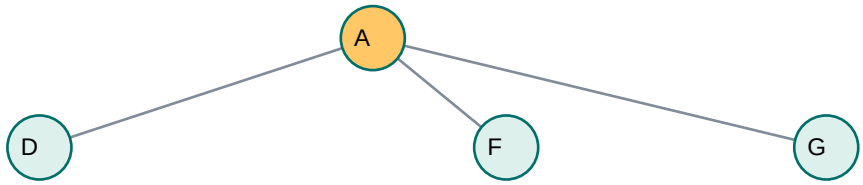
Test 1.3. Una heurística es innecesaria para que una búsqueda sea óptima: BFS con costes constantes y UCS con costes positivos son dos ejemplos. Un coste constante por paso tampoco convierte DFS en óptima ni impide que una rama infinita la bloquee.

Solucionario, ejercicios 1.3 y 4. El análisis de DFS en el manual sale del solucionario y de la referencia de la guía.

DFS · ejecución visual

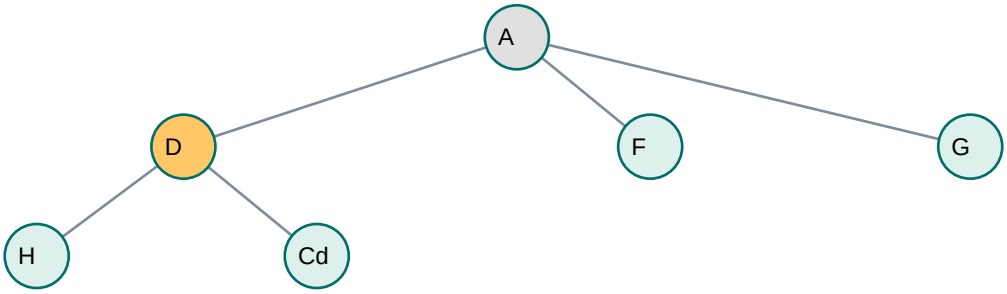
Gris: procesado. Verde claro: pendiente. Naranja: nodo del paso. Rojo: meta. C y K se duplican por sus caminos.

Paso 1 · extraer A



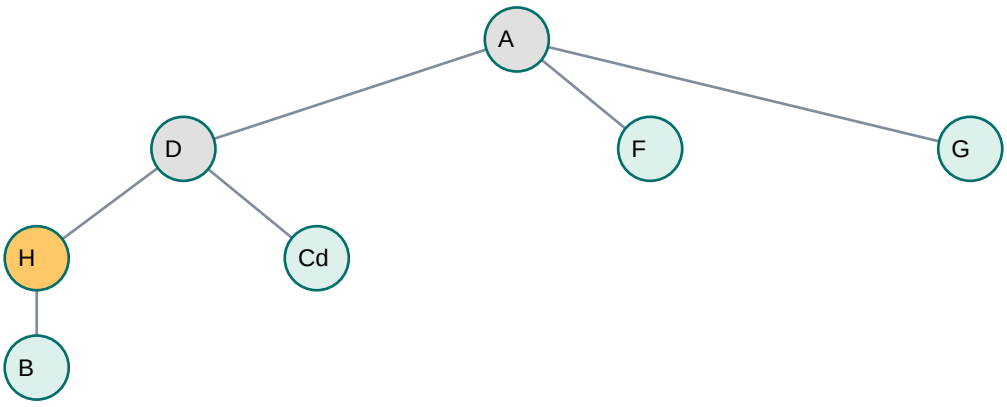
ABIERTA: D F G

Paso 2 · extraer D



ABIERTA: H Cd F G

Paso 3 · extraer H

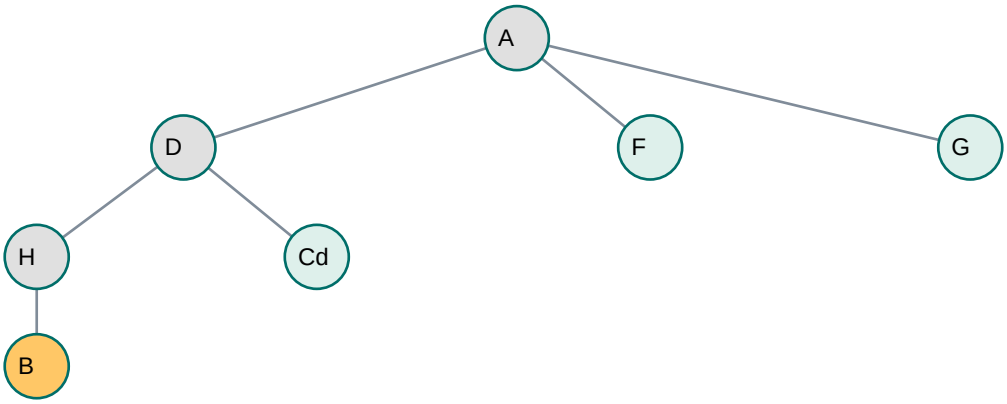


ABIERTA: B Cd F G

DFS · ejecución visual

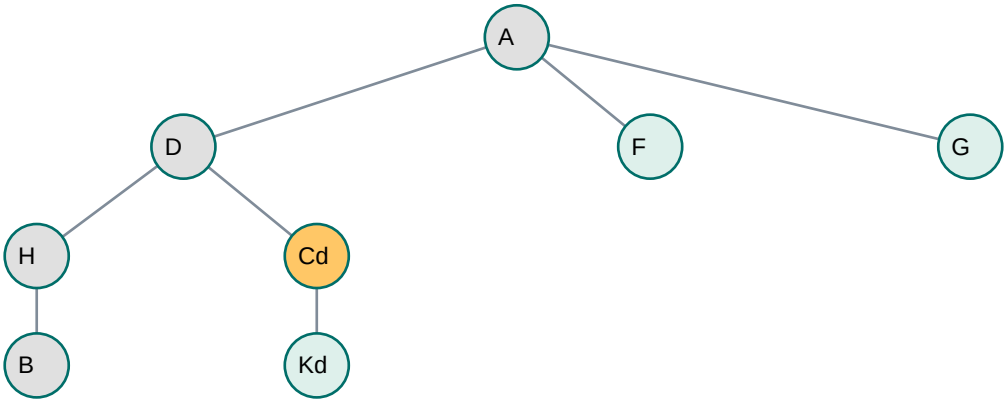
Gris: procesado. Verde claro: pendiente. Naranja: nodo del paso. Rojo: meta. C y K se duplican por sus caminos.

Paso 4 · extraer B



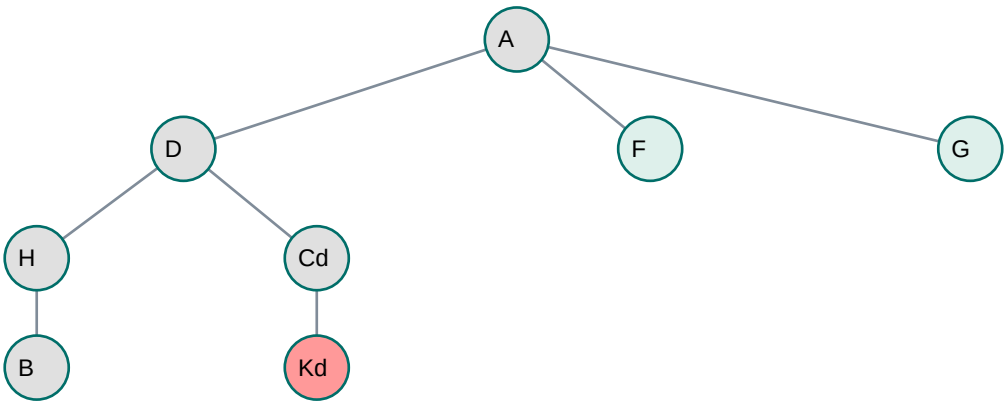
ABIERTA: Cd F G

Paso 5 · extraer Cd



ABIERTA: Kd F G

Paso 6 · extraer Kd



ABIERTA: F G

Meta aceptada. Se reconstruye el camino mediante los padres.

Filtrar al generar

Política	Qué rechaza	Qué puede seguir apareciendo
Sin filtro	Nada	Ciclos y subárboles duplicados.
Ciclos simples	Estado del padre del nodo que se expande.	A→B→C→A y convergencias.
Ciclos generales	Cualquier estado del camino actual, incluido el nodo que se expande si hay bucles a sí mismo.	Dos rutas distintas hacia un mismo estado.
Todos los repetidos	Cualquier estado registrado, pendiente o ya extraído.	Una copia por estado en BFS o DFS sin límite de profundidad.

Para BFS con coste unitario, marca el estado **al insertarlo** en abierta. Así no entran dos copias pendientes y se conserva el camino menos profundo. Para UCS, si haces búsqueda en grafo, conserva el mejor g conocido y actualiza una entrada pendiente cuando mejora. Puedes descartar las entradas antiguas al extraer. El primer descubrimiento podría ser por un camino caro.

Ejercicio 5 · BFS de las jarras con filtro global

Paso	ABIERTA antes de extraer	Paso	ABIERTA antes de extraer
1	(0,0)	8	(3,3) · (0,1)
2	(4,0) · (0,3)	9	(0,1) · (4,2)
3	(0,3) · (4,3) · (1,3)	10	(4,2) · (4,1)
4	(4,3) · (1,3) · (3,0)	11	(4,1) · (0,2)
5	(1,3) · (3,0)	12	(0,2) · (2,3)
6	(3,0) · (1,0)	13	(2,3) · (2,0)
7	(1,0) · (3,3)		

Cómo leer los pasos. Al extraer (4,0), descarta (0,0) por visitado y encola (4,3) y (1,3) tras (0,3). Al extraer (4,3) no añades nada: (0,3) y (4,0) ya se habían registrado. Al extraer (4,1), el trasvase a Tres genera (2,3), que queda detrás de (0,2). Al extraer (0,2), se añade (2,0) al final.

Primera meta: (2,3), con plan (0,0)→(4,0)→(1,3)→(1,0)→(0,1)→(4,1)→(2,3). Operadores 1,6,4,8,1,6. Tiene coste 6, igual que el plan hacia (2,0) del ejercicio 2. La fila 13 del original conserva (0,2) por errata. Su árbol sí muestra (2,0).

Diapositiva 21. Solucionario, ejercicios 2 y 5. Abierta se muestra antes de extraer, como en el anexo.

El cruce del río

Ejercicio 6. El estado es el par de conjuntos de las orillas. Basta escribir el conjunto izquierdo, ya que el derecho es su complemento. h = hombre, o = oveja, l = lobo, c = col. Inicial holc, meta ∅. Cada viaje mueve h y, opcionalmente, un acompañante de su orilla. Prueba en orden o, c, solo, l.

Filtro de legalidad: tras mover, descarta el resultado si en la orilla sin h quedan o+l u o+c. **Filtro de repetidos:** entre los estados legales conserva solo los que nunca se han insertado. La oveja es el único primer viaje legal. Desde **lc** puede regresar con ella al inicial (se descarta) o regresar solo a **hlc**.

Paso	ABIERTA de BFS	ABIERTA de DFS
1	holc	holc
2	lc	lc
3	hlc	hlc
4	l · c	l · c
5	c · hol	hol · c
6	hol · hoc	o · c
7	hoc · o	hoc · ho · c
8	o	ho · c
9	ho	∅ · c
10	∅	Fin

Dónde se separan BFS y DFS

Desde **hlc**, llevar la col deja **l** a la izquierda y llevar el lobo deja **c**. BFS toma ambas ramas por niveles. **hol** y **hoc** pueden generar **o**, pero solo lo conserva el primer padre que lo inserta. DFS deja **c** pendiente. Cuando llega a **o**, prueba antes el regreso con la col y obtiene **hoc**. Desde **hoc** los retornos legales llevan a estados registrados. Entonces retrocede y extrae **ho**.

> **Plan de ambas trazas.** holc→lc→hlc→l→hol→o→ho→∅. Cruza con oveja, vuelve solo, cruza con col, vuelve con oveja, cruza con lobo, vuelve solo y cruza con oveja. Siete cruces. BFS garantiza el mínimo número de viajes.

Sin filtro. DFS alterna indefinidamente holc→lc→holc porque prefiere viajar con la oveja. BFS sigue siendo completa: hay un número finito de caminos hasta cada nivel y la meta está a profundidad 7.

Solucionario, ejercicio 6. Las cabezas de la tabla reproducen el orden numerado en los dos árboles originales.

06 / COSTE UNIFORME

UCS · Primero por coste

g(raíz)=0 y **g(hijo)=g(padre)+c(padre,hijo)**. Cada nodo lleva el coste de **su camino**. Inserta por g ascendente y comprueba la meta al extraer el mínimo. Tus apuntes destacan que el orden se decide por g. «Uniforme» se refiere al orden por coste acumulado, y admite operadores con costes distintos.

Por qué funciona

• **Orden.** Con arcos positivos, los hijos cuestan más que el padre. Las extracciones tienen g no decreciente. Las fronteras por coste de la diapositiva 35 representan umbrales acumulados desde el origen.

• **Completitud.** Si b es finito y cada paso cuesta al menos ε>0 (ε=1 con enteros positivos), hasta el coste C* de una meta solo hay caminos de longitud ≤⌊C*/ε⌋. Hay un número finito de nodos de ese coste o menor. Una cadena infinita de costes positivos sin cota inferior puede romper esta garantía.

• **Optimalidad.** Si al extraer una meta hubiera otra solución más barata, esa ruta tendría un prefijo aún pendiente con g menor. Ese prefijo habría salido primero. En el modelo en árbol, se conservan rutas distintas al mismo estado según el filtro. Con filtrado global ponderado, una mejora de g debe poder sustituir un camino pendiente.

Recursos. Sea L=⌊C*/ε⌋. Hasta la primera meta pueden procesarse nodos de profundidad hasta L y generarse hijos en L+1. En árbol, con b≥2, el límite por número de nodos es $O(b^{L+1})$ para tiempo y espacio. La cola de prioridad añade su propio coste de inserción y extracción.

Rumanía · Menos saltos puede costar más

Arad→Sibiu→Făgăraș→Bucarest cuesta **140+99+211=450** (3 arcos). Arad→Sibiu→Rîmnicu→Pitești→Bucarest cuesta **140+80+97+101=418** (4 arcos). BFS prioriza 3 arcos. UCS prioriza el coste. En la diapositiva 34, sin filtro, las primeras cabezas son A0, Z75, T118 y S140. Al expandir Z aparece A150 como ruta de vuelta. Con filtro de ciclos simples se quitaría.

Ejercicio 7 · De Caiova a Făgăraș

Filtra **solo el retorno al padre**. Desde C_0, añade D_{120}, P_{138} y R_{146}. D_{120} genera M_{195} (vuelta a C descartada). P_{138} genera R_{235} y B_{239}. **R_{146} y R_{235}** se conservan porque representan caminos diferentes. R_{146} genera S_{226} y P_{243}. Desde S_{226} se genera F_{325}, pero todavía quedan nodos de menor coste, entre ellos U_{324}.

Resultado. La primera meta extraída es F_{325}. Sus padres dan C→R→S→F, coste 146+80+99=325. Los nodos F_{450} y F_{414} son otros planes. F_{325} se acepta en la extracción 13, después de expandir U_{324}.

Diapositivas 31–36. Solucionario, ejercicio 7. El encabezado de su última página dice «Solución 6», aunque la traza corresponde al 7.

Coste uniforme · mapa y árbol

Figuras del material docente. Lee cada panel junto a la explicación indicada.

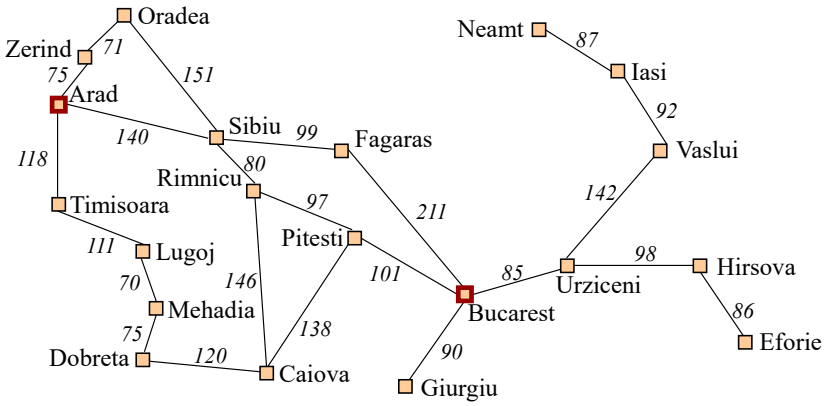
Problema de encontrar rutas

Estado: estancia en una ciudad

Coste de un operador: distancia por carretera a la ciudad vecina

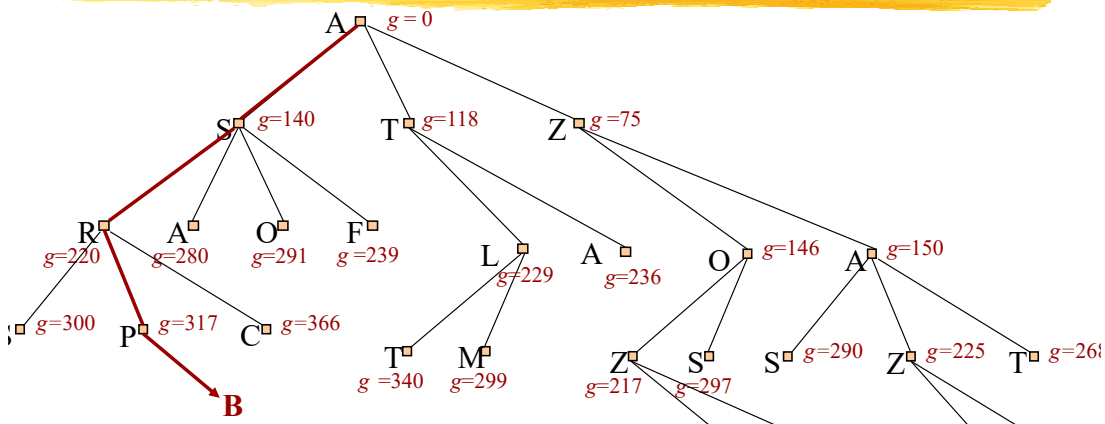
Operadores: ir a una ciudad vecina

Coste de un plan: suma de distancias entre las ciudades visitadas



Cada ciudad es un estado. Cada carretera aporta un coste. Para sumar g, sigue las carreteras del camino concreto.

Ejemplo: Búsqueda de coste uniforme



Lista *abierta*:

1. (A,0)
2. **(Z,75), (T,118), (S,140)**
3. (T,118), (S,140), **(O,146), (A,150)**
4. (S,140), (O,146), (A,150), **(L,229), (A,236)**
5. (O,146), (A,150), **(R,220)**, (L,229), (A,236), **(F,239), (A,280), (O,291)**

…

El mismo estado A aparece con costes diferentes. La prioridad de extracción depende de g y puede saltar entre ramas.

Traza de Caiova a Făgăraș

Paso	ABIERTA antes de extraer · cabeza a la izquierda
1	C0
2	D120 · P138 · R146
3	P138 · R146 · M195
4	R146 · M195 · R235 · B239
5	M195 · S226 · R235 · B239 · P243
6	S226 · R235 · B239 · P243 · L265
7	R235 · B239 · P243 · L265 · F325 · A366 · O377
8	B239 · P243 · L265 · S315 · F325 · A366 · O377 · C381
9	P243 · L265 · S315 · U324 · F325 · G329 · A366 · O377 · C381 · F450
10	L265 · S315 · U324 · F325 · G329 · B344 · A366 · O377 · C381 · C381 · F450
11	S315 · U324 · F325 · G329 · B344 · A366 · T376 · O377 · C381 · C381 · F450
12	U324 · F325 · G329 · B344 · A366 · T376 · O377 · C381 · C381 · F414 · F450 · A455 · O466
13	F325 · G329 · B344 · A366 · T376 · O377 · C381 · C381 · F414 · H422 · F450 · A455 · O466 · V466

Cómo reconstruir el árbol. Entre dos filas, elimina la cabeza extraída. Cada entrada nueva es un hijo de esa cabeza, con el peso del arco igual a la diferencia de g. Respeta la multiplicidad: en el paso 8 se genera C381 desde R235 y en el paso 10 aparece otro C381 desde P243. Un empate solo cambia el orden si las entradas empatadas llegan a extraerse. Aquí ambas copias de C381 quedan pendientes al hallar la meta.

Fuentes y alcance

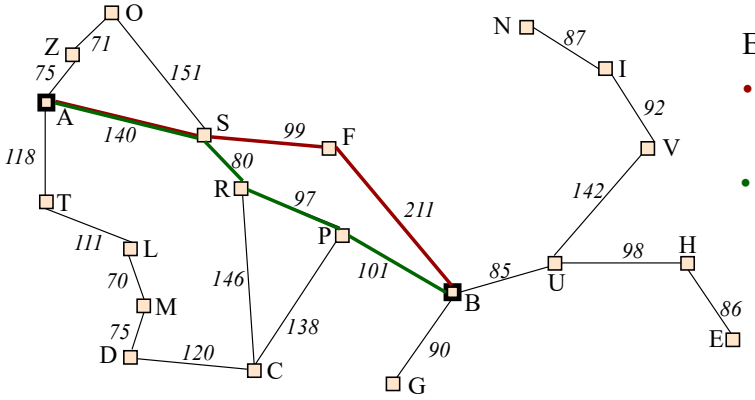
Material del curso. Guía Módulos I y II, diapositivas 1–36, hoja resuelta del tema 2 y foto de apuntes. La guía señala Russell y Norvig (2.ª ed., 2003), §§3.1–3.3, §3.4 parcial y §3.5. Como complemento, Nilsson (2001), cap. 8. El alcance de las estrategias se toma de las diapositivas y ejercicios.

Anexo desde la página siguiente. Se adjuntan las nueve páginas originales, con sus enunciados, diagramas y soluciones. Las trazas explicadas en el cuerpo aplican la prueba de meta de la diapositiva 20.

Coste uniforme · frente de búsqueda

Figuras del material docente. Lee cada panel junto a la explicación indicada.

Problema de encontrar rutas: ejemplo



Ejemplo:

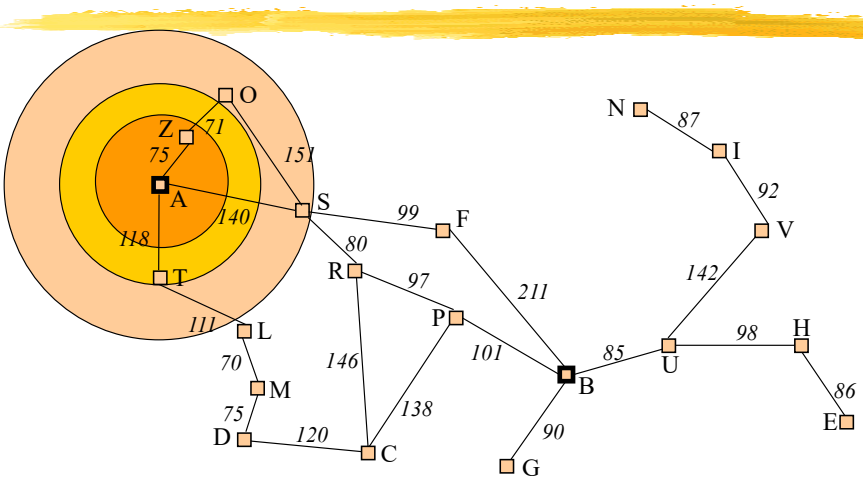
- p_1 = A-S-F-B
  c(p_1) = 450
- p_2 = A-S-R-P-B
  c(p_2) = 418

Problema:

- La búsqueda en amplitud encuentra el nodo meta de menor profundidad; éste puede *no* ser el nodo meta de coste mínimo
- prof.(B_{p1}) = 3 < 4 = prof.(B_{p2}) / c($p1$) = 450 > 418=c($p2$)

Tres arcos cuestan 450, mientras que cuatro cuestan 418. Profundidad y coste inducen órdenes distintos.

Lógica de la búsqueda de coste uniforme



$g = 80$ $g = 120$ $g = 160$

Los umbrales de g crecen desde el origen. El siguiente nodo elegido es el pendiente con menor coste acumulado.

Grupo de Inteligencia Artificial
Universidad Rey Juan Carlos

Hoja de Problemas Tema 2
Búsqueda no-informada

Universidad Rey Juan Carlos

Ejercicio 1:

1.1. El enfoque de los Agentes Inteligentes concibe el objetivo de la Inteligencia Artificial como el intento de construir sistemas

(a) que "actúen" como las personas.

(b) que "actúen" de forma eficiente (racional) en su entorno.

(c) que "piensen" como las personas (tipo General Problem Solver, GPS).

(d) Ninguna de las respuestas es correcta.

1.2. ¿Para cual(es) de las siguientes tareas pueden construirse agentes basados en algoritmos de búsqueda en espacios de estados (p.e. la búsqueda de coste uniforme)?

(a) Encontrar una solución al problema de las Torres de Hanoi.

(b) Gestionar el tráfico rodado en una red de autopistas urbanas.

(c) Conducir un taxi en las calles de Barcelona.

(d) Jugar a las cuatro en raya contra un jugador humano.

1.3. ¿Cuáles de las siguientes afirmaciones acerca de los algoritmos de búsqueda no informados es (son) cierta(s)?

(a) Los algoritmos de búsqueda no informados requieren de información heurística para que sean óptimos.

(b) La búsqueda en amplitud es óptima y completa siempre y cuando el coste de los operadores sea constante.

(c) La búsqueda en profundidad es óptima y completa siempre que el coste de los operadores sea constante.

(d) Tanto la complejidad en tiempo como la complejidad en espacio de la búsqueda en amplitud se pueden expresar en función del número de nodos expandidos

1.4. Contemple el problema de búsqueda de la figura 1. ¿Cuál es el factor de ramificación del árbol de búsqueda generado por los métodos de búsqueda en el espacio de estados, si no se filtran estados repetidos?

(a) 0.5

(b) 2

(c) 4

(d) 6

1.5. Contemple el problema de búsqueda de la figura 1. En el árbol generado por la búsqueda en amplitud, si no se filtran estados repetidos, ¿cuántos nodos hay a nivel de profundidad k (suponiendo que la raíz tiene nivel 0, es decir $k \geq 0$)

(a) $k/4$

(b) 4^k

(c) $4*k^2$

(d) k^4

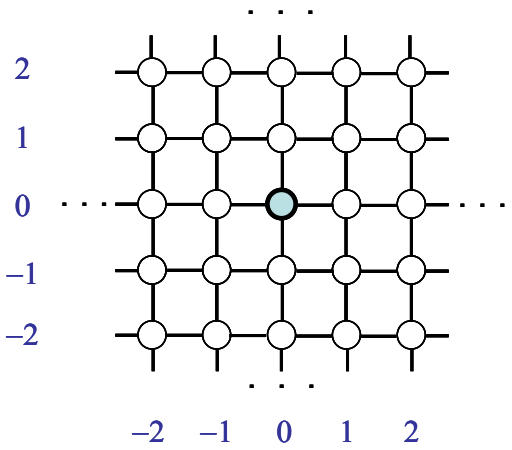


Figura 1. *Red 2D regular e infinito.*

- *Estado inicial (0,0).*
- *Estado meta (x,y); x,y ∈Z.*
- *Movimiento entre estados directamente conectados a coste 1*

Ejercicio 2:

En una mesa se encuentran dos jarras, una con una capacidad de 3 litros (llamada *Tres*), y la otra con una capacidad de 4 litros (llamada *Cuatro*). Inicialmente, *Tres* y *Cuatro* están vacías. Cualquiera de ellas puede llenarse con el agua de un grifo *G*. Asimismo, el contenido tanto de *Tres* como de *Cuatro* puede vaciarse en una pila *P*. Es posible echar todo el agua de una jarra a la otra. No se dispone de dispositivos de medición adicionales. Se trata de encontrar una secuencia de operadores que deje exactamente dos litros de agua en *Cuatro*.

a) Modele este problema como un problema de búsqueda. Con tal fin, defina el estado inicial, el conjunto de estados meta, los operadores (especificando sus precondiciones y postcondiciones), así como el coste de cada operador.

b) Caracterice el conocimiento a priori del agente de resolución del problema correspondiente? Facilite ejemplos de los resultados de la función expandir.

c) Encuentre una solución al problema.

Solución a)

Información relevante: sólo la cantidad de litros en cada jarra
Se modela un estado como un par (x, y), significando que hay x litros en *Cuatro* e y litros en *Tres*

- S = {0, 1, 2, 3, 4} x {0, 1, 2, 3}
- $s_0 = (0, 0)$
- s_m = {(2,0), (2,1), (2,2), (2,3)} ó: $meta?((x,y)) = \begin{cases} true\ si\ x = 2 \\ false\ de\ lo\ contrario \end{cases}$

Operadores:

- op_1: Llenar *Cuatro*
  - $(x, y) \rightarrow (4, y)$
  - Prec: $x < 4$
- op_2: Llenar *Tres*
  - $(x, y) \rightarrow (x, 3)$
  - Prec: $y < 3$
- op_3: Vaciar *Cuatro*
  - $(x, y) \rightarrow (0, y)$
  - Prec: $x > 0$
- op_4: Vaciar *Tres*
  - $(x, y) \rightarrow (x, 0)$
  - Prec: $y > 0$
- op_5: Llenar *Cuatro* desde *Tres*
  - $(x, y) \rightarrow (4, y - (4 - x))$
  - Prec: $x + y \geq 4, x < 4, [y > 0]$
- op_6: Llenar *Tres* desde *Cuatro*
  - $(x, y) \rightarrow (x - (3 - y), 3)$
  - Prec: $x + y \geq 3, y < 3, [x > 0]$
- op_7: Vaciar *Tres* en *Cuatro*
  - $(x, y) \rightarrow (x + y, 0)$
  - Prec: $x + y < 4, y > 0$
- op_8: Vaciar *Cuatro* en *Tres*
  - $(x, y) \rightarrow (0, x + y)$
  - Prec: $x + y < 3, x > 0$

Coste(op_i) = 1

Solución b)

Ejemplos función *expandir*:

- *expandir*((0,0)) = {(4,0) [op1] , (0,3) [op2]}
- *expandir*((4,0)) = {(0,0) [op3], (4,3) [op2], (1,3) [op6]}
- *expandir*((1,3)) = {(4,3) [op1], (0,3) [op3], (1,0) [op4], (4,0) [op5]}

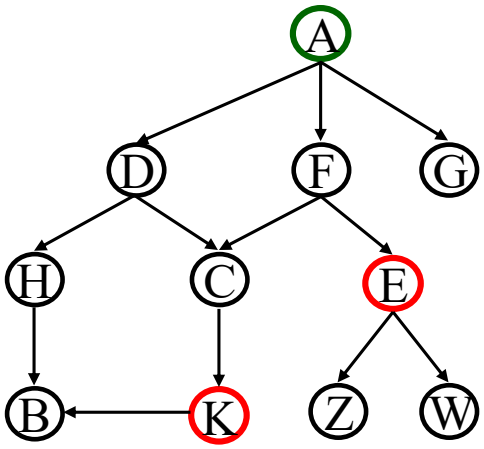
Los operadores están *implícitos* en la función *expandir* (de hecho, se pueden usar para "computarla", en vez de almacenar *expandir* en la memoria en forma de tabla).

Solución c)

P.e.: (0,0) –op_2–> (0,3) –op_7–> (3,0) –op_2–> (3,3) –op_5–> (4,2) –op_3–> (0,2) –op_7–> (2,0)

Ejercicio 3:

El grafo que se muestra al lado determina un problema de búsqueda. Cada nodo representa un estado; los arcos modelan la aplicación de operadores. Suponga que *A* es el estado inicial y que *K* y *E* son estados meta

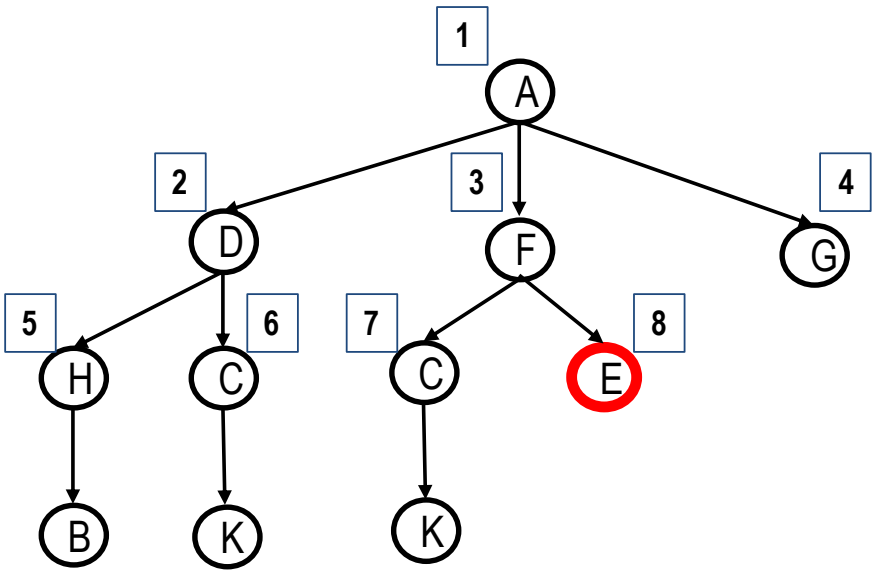


a) Desarrolle el árbol de búsqueda que genera la búsqueda en amplitud, *sin* filtrar estados repetidos. ¿Cuál de los nodos meta se encuentra primero?

b) Indique el orden en que se expanden los nodos

c) Ponga el estado de la lista abierta en cada paso del algoritmo

Solución 3a y b)



Solución 3c)

1. A
2. D F G
3. F G H C
4. G H C C E
5. H C C E
6. C C E B
7. C E B K
8. E B K K

Ejercicio 4:

a) ¿Cómo se podría instanciar el algoritmo de búsqueda genérico (de la diapositiva 20) para implementar una búsqueda en profundidad?

b) Para el grafo del ejercicio 3, desarrolle el árbol de búsqueda que genera la búsqueda en profundidad. Indique el orden en que se expanden los nodos ¿Cuál de los nodos meta se encuentra primero?

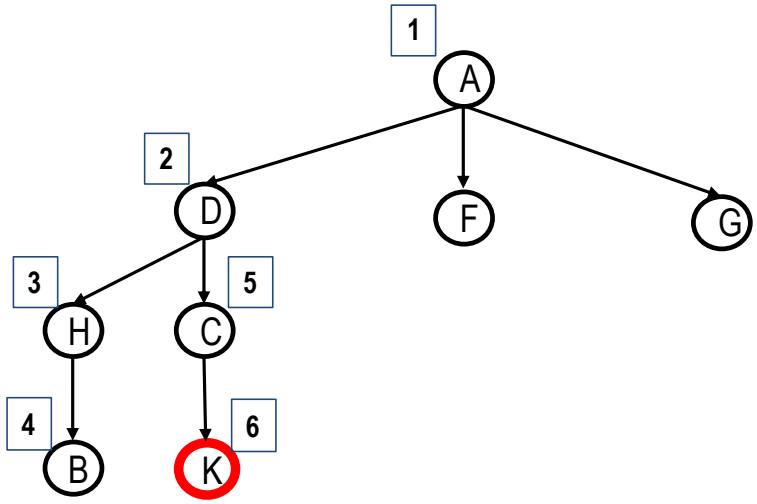
c) Haga un análisis de complejidad de dicho algoritmo, similar al de la diapositiva 28, asumiendo un límite de profundidad d^* fijado a priori.

Solución 4a):

Para implementar una búsqueda en profundidad, entre las hojas del árbol de búsqueda hay que elegir aquellos de mayor profundidad. Esto es equivalente a preferir las hojas que menos tiempo llevan en la lista *abierta*. Es decir, *abierta* ha de funcionar como una estructura LIFO. Para que *abierta* funcione como una pila (con comportamiento LIFO), y puesto que la función *primero* quita un elemento de la cabeza de la lista, también habrá que insertar nuevos elementos en la cabeza, e.d. la última línea del algoritmo sería:

```
ordInsertar(n,abierta,cabeza)
```

Solución 4b):



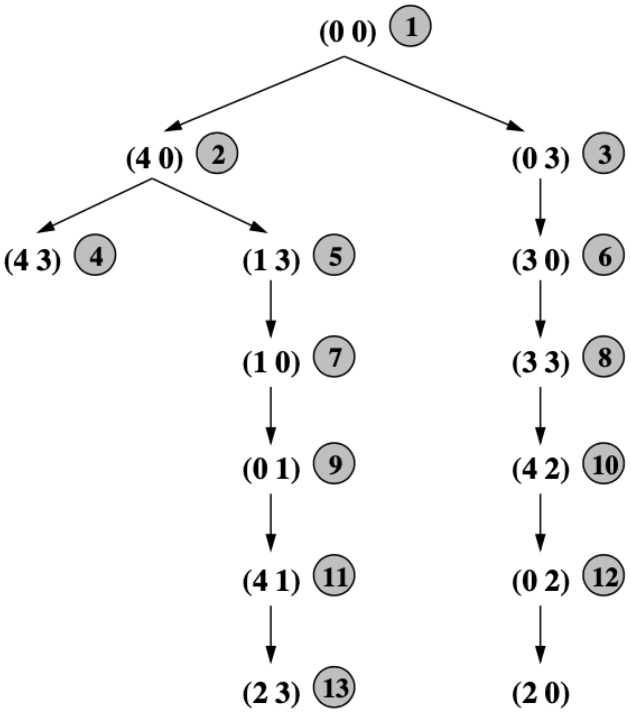
Solución 4c):

En general, la búsqueda en profundidad no es óptima (como en el ejemplo), y tampoco completa (salvo que haya un límite de profundidad d^* a priori, en el peor caso búsqueda en profundidad se puede perder en ramas infinitas). Respecto a la complejidad en tiempo, en el mejor caso es O(d) (se expande directamente el camino que lleva al mejor nodo meta de profundidad d), y en el peor caso O(b^{d^*}) (igual que la búsqueda en amplitud). Respecto a la complejidad en espacio, sobre la base de nuestra implementación (solución 5a), es igual que la de la búsqueda en amplitud. Sin embargo, una implementación más eficiente (p.e. recursiva) tendría una complejidad en espacio de O(b*d) (se almacenan los nodos de un "camino actual" y sus hermanos). Esto se debe a que, una vez que se expanda un nodo sin hijos (como el nodo *B* que se expande en el paso 4), es posible abandonar el camino actual, y liberar la memoria correspondiente (en el ejemplo los nodos *B* y *H*).

Ejercicio 5:

Considere el problema de las jarras del ejercicio 2. Simule la estrategia de búsqueda en amplitud para este problema, asumiendo que se filtran *todos los estados repetidos*. Indique el orden en que se expanden los nodos, y ponga el estado de la lista abierta en cada paso del algoritmo.

Solución 5:



Lista abierta:

1. **(0,0)**
2. **(4,0), (0,3)**
3. (0,3), **(4,3), (1,3)**
4. (4,3), (1,3), **(3,0)**
5. (1,3), (3,0)
6. (3,0), **(1,0)**
7. (1,0), **(3,3)**
8. (3,3), **(0,1)**
9. (0,1), **(4,2)**
10. (4,2), **(4,1)**
11. (4,1), **(0,2)**
12. (0,2), **(2,3)**
13. (2,3), **(0,2)**

Ejercicio 6:

Considere el siguiente problema:

> *Un hombre se encuentra en la orilla izquierda de un río junto con un lobo, una oveja y una col. Quiere cruzar el río llevando consigo el lobo, la oveja y la col. En la barca sólo hay dos plazas, una de las cuales debe ir ocupada por el hombre. Cada uno de los restantes pasajeros (lobo, oveja, col) ocupa una plaza, de tal modo que sólo uno puede acompañar al hombre en cada viaje. Además, no puede dejar solos en una orilla al lobo con la oveja, ni a la oveja con la col (ni los tres), porque la primera se comería a la segunda en cada paso*

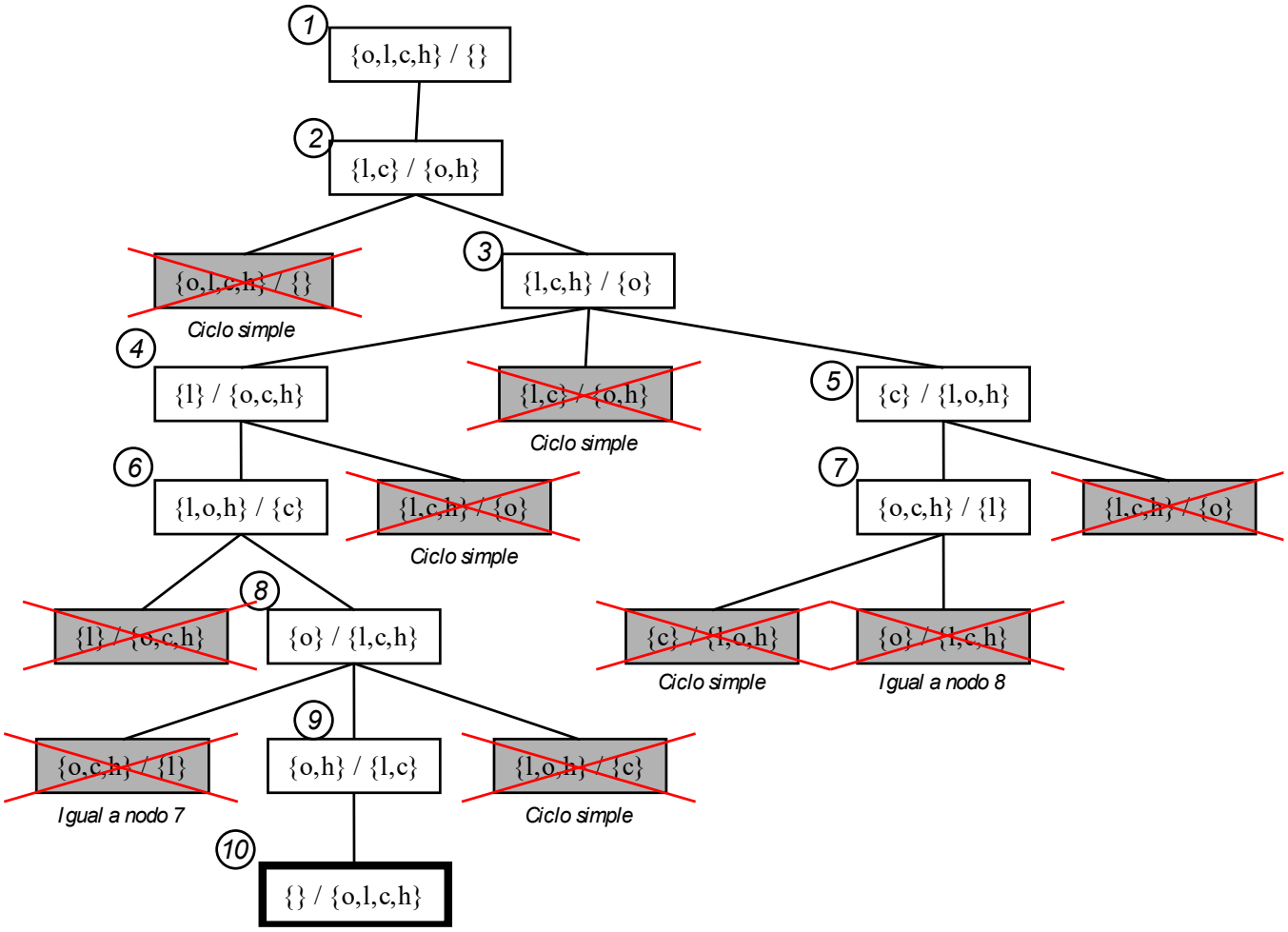
Suponga que se modela el problema como un espacio de estados, donde cada estado se describe como un par de conjuntos, indicando quien(es) se encuentran en cada orilla del río (c = col; o = oveja; l = lobo; h = hombre). Por tanto, el estado inicial del problema sería $(\{c,o,l,h\},\{\})$, y el estado meta $(\{\},\{c,o,l,h\})$.

a) Simule la estrategia de *búsqueda en amplitud* para este problema, asumiendo que se filtran *todos* los estados repetidos. Expanda los sucesores de los nodos siguiendo las preferencias del hombre, las cuales se ordenan (de mayor a menor) como sigue: viajar con la oveja, viajar con la col, viajar sólo, viajar con el lobo. Dibuje el árbol de búsqueda correspondiente e indique el orden en el que se exploran los nodos.

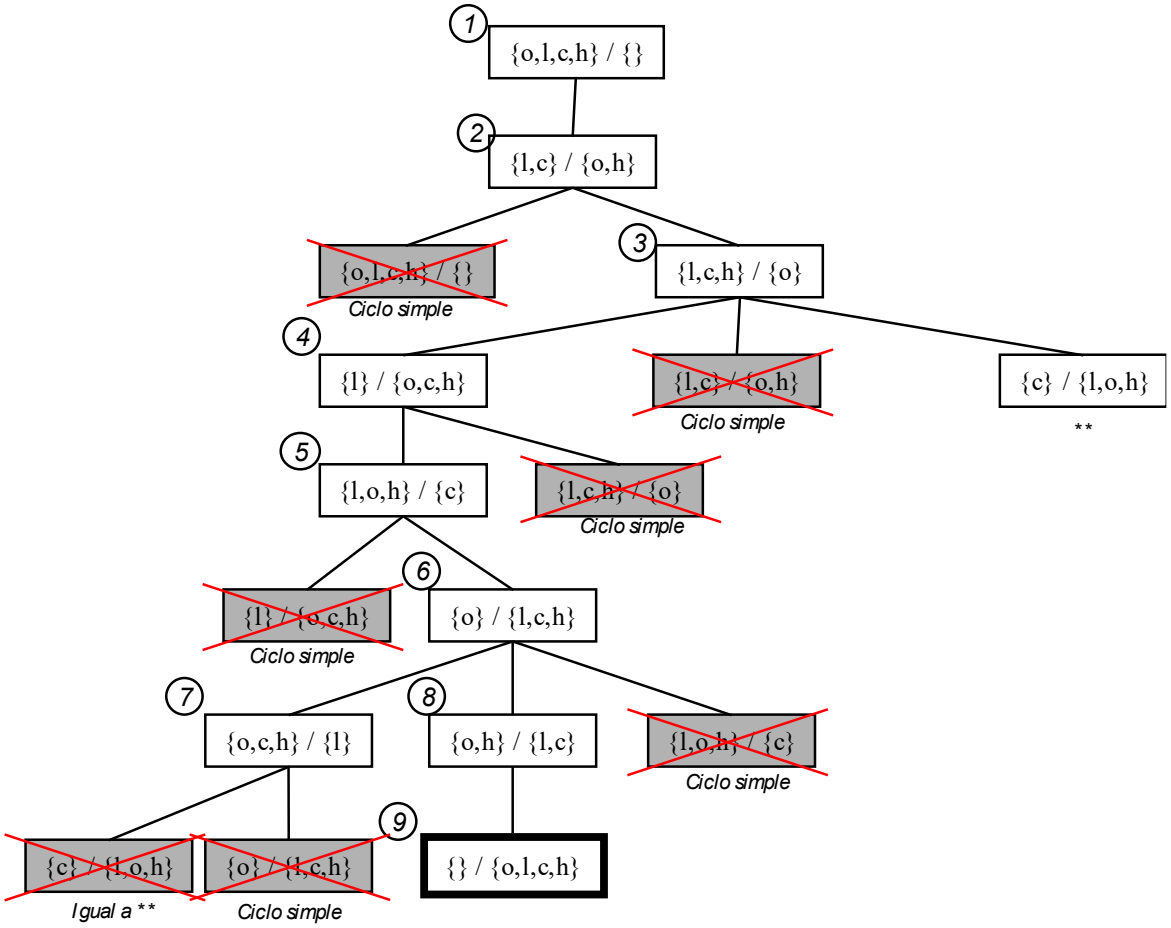
b) Simule ahora la estrategia de *búsqueda en profundidad* para este problema. Dibuje el árbol de búsqueda correspondiente e indique el orden en el que se exploran los nodos, asumiendo que se filtran *todos* los estados repetidos y siguiendo las preferencias del apartado anterior.

c) Suponga ahora que no se filtra ningún estado repetido. ¿La búsqueda en amplitud y la búsqueda en profundidad seguirían siendo completas? Razone brevemente su respuesta.

Solución 6a



Solución 6b



Solución 6c

Si se dejaran de filtrar estados repetidos, en principio se podrían dar secuencias infinitas de acciones. Por tanto, la búsqueda en profundidad ya no sería completa. En el ejemplo del apartado b), el proceso de búsqueda se metería en la rama infinita {o,l,c,h} / {} → {l,c} / {o,h} → {o,l,c,h} / {} → ...
Por otra parte, la búsqueda en amplitud del apartado a) seguiría siendo completa. De hecho, siempre es completa (si el coste de los operadores es constante), aun cuando el conjunto de estados posibles es infinito.

Ejercicio 7:

Suponga que en la red de carreteras de Rumania presentada en clase (diapositiva 31) nuestra agente se encuentra en Caiva (C) y desea trasladarse a Fagaras (F)

a) Desarrolle el árbol de búsqueda que genera la *búsqueda de coste uniforme*, indicando los valores de *g* para cada nodo. Suponga que se filtran *ciclos simples*.

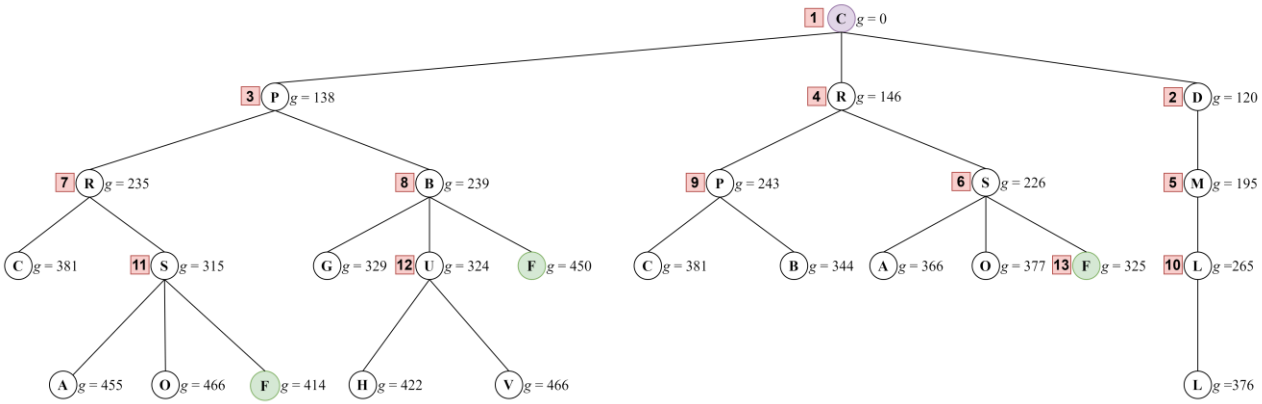
b) Indique el orden en que se expanden los nodos

c) Ponga el estado de la lista abierta en cada paso del algoritmo

Solución 6:

En lo que sigue, se supone que se filtran ciclos simples.

a y b):



c)

1. (C,0)
2. (D,120) (P,138) (R,146)
3. (P,138) (R,146) (M,195)
4. (R,146) (M,195) (R,235) (B,239)
5. (M,195) (S,226) (R,235) (B,239) (P,243)
6. (S,226) (R,235) (B,239) (P,243) (L,265)
7. (R,235) (B,239) (P,243) (L,265) (F,325) (A,366) (O,377)
8. (B,239) (P,243) (L,265) (S,315) (F,325) (A,366) (O,377) (C,381)
9. (P,243) (L,265) (S,315) (U, 324) (F,325) (G,329) (A,366) (O,377) (C,381) (F,450)
10. (L,265) (S,315) (U, 324) (F,325) (G,329) (B,344) (A,366) (O,377) (C,381) (C,381) (F,450)
11. (S,315) (U, 324) (F,325) (G,329) (B,344) (A,366) (T,376) (O,377) (C,381) (C,381) (F,450)
12. (U,324) (F,325) (G,329) (B,344) (A,366) (T,376) (O,377) (C,381) (C,381) (F,414) (F,450) (A,455) (O,466)
13. (F,325) (G,329) (B,344) (A,366) (T,376) (O,377) (C,381) (C,381) (F,414) (H,422) (F,450) (A,455) (O,466) (V,466)